

AMERICAN UNIVERSITY
OF PHNOM PENH
STUDY LOCALLY. LIVE GLOBALLY.

School of Digital Technologies

ITM-360: Artificial Intelligence

Object Detection: YOLO

Lecturer: Kuntha Pin (Ph.D Eng.)

Overview of today's lecture

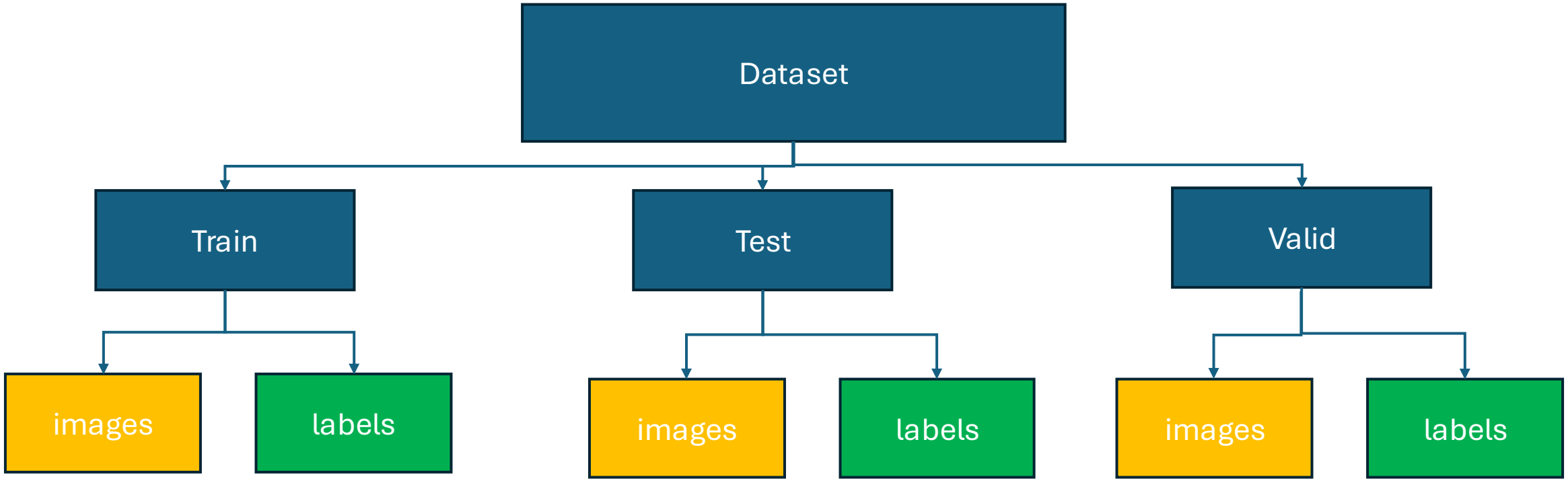
1. Introduction
2. Understanding the Dataset
3. Data Preparation
4. Fine-tuning YOLO
5. Prediction
6. Practice

1. Introduction

- In this study, a deep learning model based on YOLO is employed to detect and classify blood cell images into three categories.
- The YOLO model was originally trained on the COCO dataset for general object detection.
- However, since the COCO dataset does not include blood cell images, the pretrained YOLO model must be fine-tuned on our custom dataset to adapt it to the target task.

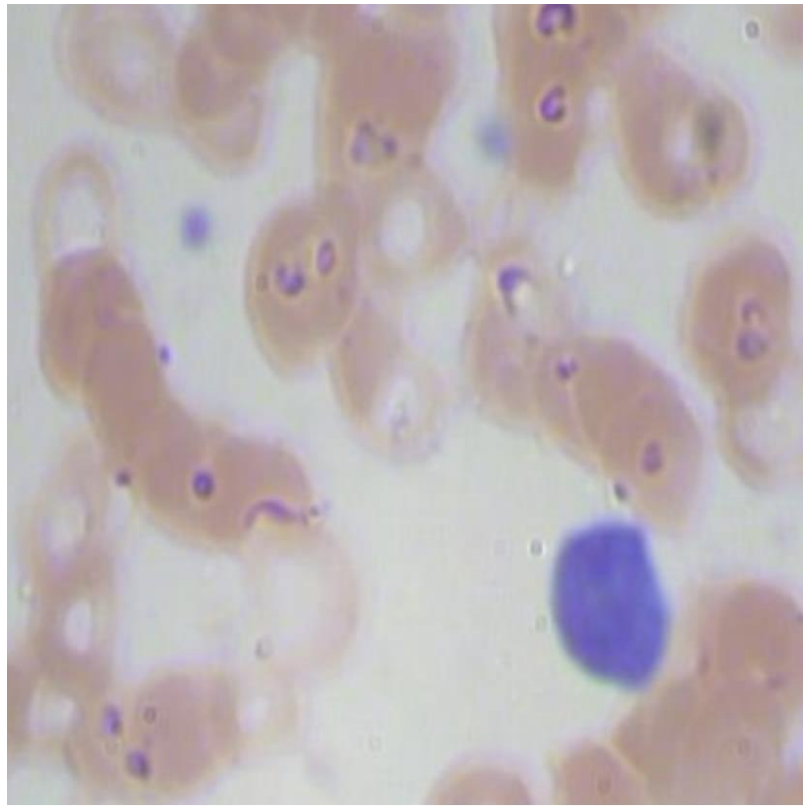
2. Understanding the Dataset

The blood cell dataset is avalialbe in Kaggle daset: https://www.kaggle.com/datasets/adhoppin/blood-cell-detection-datatset?select=train



2. Understanding the Dataset

Example of image and it's label file.



images

```
2 0.7403846153846154 0.7620192307692307 0.18389423076923078 0.25600961538461536
1 0.31971153846153844 0.8786057692307693 0.15625 0.13701923076923078
1 0.22596153846153846 0.9110576923076923 0.15264423076923078 0.15985576923076922
1 0.07932692307692307 0.6466346153846154 0.125 0.20673076923076922
1 0.09254807692307693 0.7536057692307693 0.125 0.20673076923076922
1 0.08413461538461539 0.26322115384615385 0.125 0.20673076923076922
1 0.11298076923076923 0.4014423076923077 0.15625 0.21033653846153846
1 0.15745192307692307 0.5096153846153846 0.12379807692307693 0.21033653846153846
1 0.20673076923076922 0.5721153846153846 0.12379807692307693 0.21033653846153846
1 0.2439903846153846 0.5901442307692307 0.12379807692307693 0.21033653846153846
1 0.3401442307692308 0.5997596153846154 0.1466346153846154 0.21634615384615385
1 0.4879807692307692 0.4963942307692308 0.1466346153846154 0.21634615384615385
1 0.375 0.3629807692307692 0.16826923076923078 0.25841346153846156
0 0.24519230769230768 0.2848557692307692 0.052884615384615384 0.08173076923076923
0 0.609375 0.16947115384615385 0.052884615384615384 0.08173076923076923
1 0.5108173076923077 0.28125 0.14423076923076922 0.17067307692307693
1 0.8762019230769231 0.17067307692307693 0.14423076923076922 0.17067307692307693
1 0.8100961538461539 0.15985576923076922 0.14543269230769232 0.25600961538461536
1 0.9230769230769231 0.3942307692307692 0.1466346153846154 0.25600961538461536
1 0.9387019230769231 0.5264423076923077 0.12139423076923077 0.1935096153846154
1 0.8149038461538461 0.5745192307692307 0.14543269230769232 0.19591346153846154
1 0.7584134615384616 0.4963942307692308 0.14543269230769232 0.19591346153846154
1 0.6850961538461539 0.46875 0.14543269230769232 0.19591346153846154
1 0.9254807692307693 0.7896634615384616 0.14423076923076922 0.19951923076923078
1 0.9254807692307693 0.9002403846153846 0.14423076923076922 0.19471153846153846
```

labels

3. Data Preparation

Prepare custom dataset

- Construct dataset paths
- Define Bounding Box Color
- Create custom dataset in data.yaml

Load pre-train model

```
# Load a pretrained YOLOv8 model
model = YOLO("yolov8x.pt")
```

```
# Train YOLO model with custom dataset
```

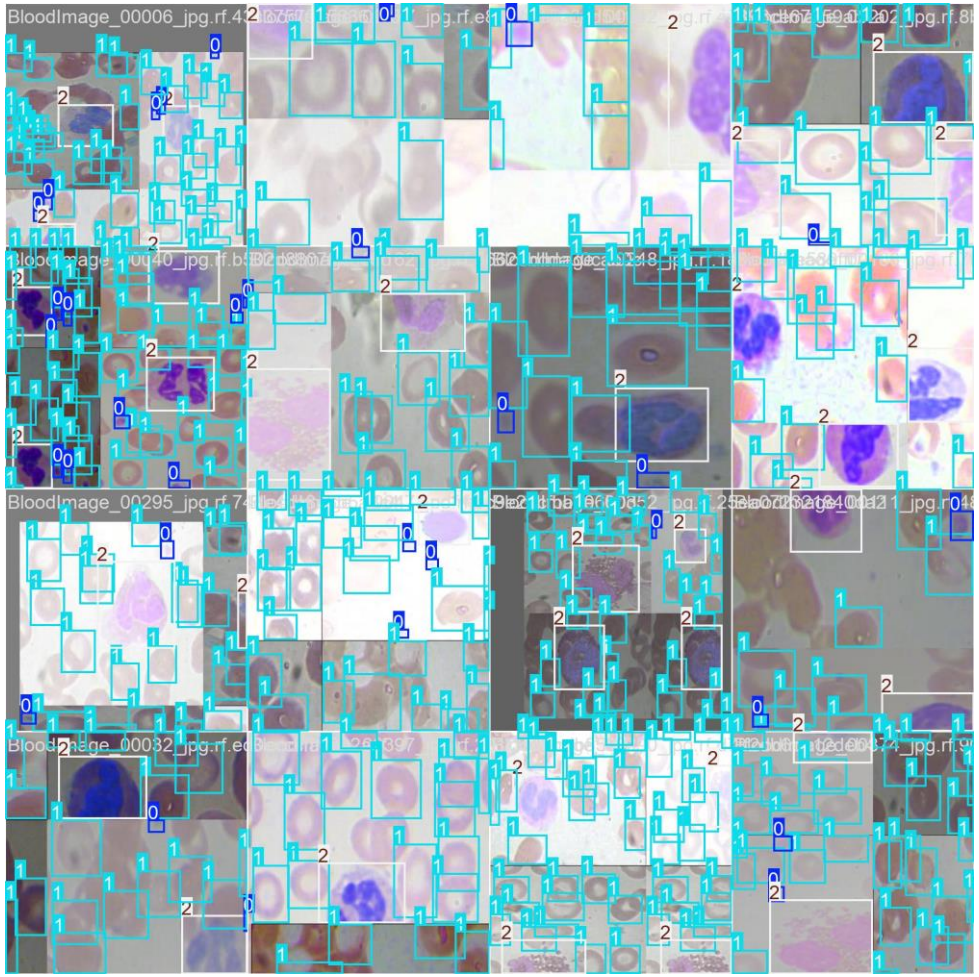
```
Ultralytics 8.3.222 🚀 Python-3.11.6 torch-2.7.1 CPU (Apple M1)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augm
Overriding model.yaml nc=80 with nc=3

                   from  n    params  module
  0                  -1  1      2320  ultralytics.nn.modules.conv.Conv
  1                  -1  1    115520  ultralytics.nn.modules.conv.Conv
  2                  -1  3    436800  ultralytics.nn.modules.block.C2f
  3                  -1  1    461440  ultralytics.nn.modules.conv.Conv
  4                  -1  6   3281920  ultralytics.nn.modules.block.C2f
  5                  -1  1   1844480  ultralytics.nn.modules.conv.Conv
  6                  -1  6  13117440  ultralytics.nn.modules.block.C2f
  7                  -1  1   3687680  ultralytics.nn.modules.conv.Conv
  8                  -1  3   6969600  ultralytics.nn.modules.block.C2f
  9                  -1  1   1025920  ultralytics.nn.modules.block.SPPF
 10                  -1  1         0  torch.nn.modules.upsampling.Upsa
```

```
!yolo task=detect mode=train model=yolov8x.pt data=data.yaml epochs= 5 imgsz= 480
```

4. Prediction

- Use image in test folder to predict objects
- Compare the bounding box label with bounding box predicted by YOLO model
- Calculate mean average precision(mAP) to measures how well the model's predicted bounding boxes match the ground truth bounding boxes



5. Practice

- Download the dataset from the provided link.
- Download the file "YOLO_Object_Detection.zip" from Canvas.
- Add code to check whether your PC is using a GPU.
- Run the notebook yolo_detect_blood_cell.ipynb.
- Modify the dataset according to your preferred task (e.g., detecting objects in a traffic dataset).
- Submit this lab assignment by Saturday.

Noted:

You can check whether your PC can access a GPU in Python using **PyTorch** or **TensorFlow**. Here's how:

Using pytorch library

```
import torch

if torch.cuda.is_available():
   print("GPU is available")
   print("GPU Name:",
torch.cuda.get_device_name(0))
else:
   print("GPU is not available, running on CPU")
```

Using tensorflow library

```
import tensorflow as tf

gpus = tf.config.list_physical_devices('GPU')
if gpus:
   print("GPU is available")
   for gpu in gpus:
      print("GPU Name:", gpu)
else:
   print("GPU is not available, running on CPU")
```

End of Class

